

1. Introducción y Arquitectura General

DateManager es una aplicación SPA (Single Page Application) construida con Angular 21 que permite a negocios gestionar sus citas en línea. La arquitectura sigue el patrón de componentes independientes (*standalone components*) sin NgModule, usando señales (*signals*) de Angular para la reactividad.

Estructura de carpetas

Ruta	Propósito
src/app/	Raíz de la aplicación
src/app/shared/	Componentes y modelos compartidos
src/app/shared/models.ts	Interfaces TypeScript del dominio
src/app/login-page/	Página de autenticación
src/app/registration-page/	Página de registro
src/app/appointments-page/	Gestión de citas (dashboard)
src/app/services-page/	Gestión de servicios del negocio
src/app/availability-page/	Configuración de disponibilidad horaria
src/app/business-profile-page/	Perfil público del negocio
src/app/booking-page/	Flujo de reserva pública (sin login)
src/app/landing-page-component/	Página de aterrizaje
src/styles.scss	Variables CSS globales y reset

2. Modelo de Datos — src/app/shared/models.ts

Todas las interfaces TypeScript que describen el dominio de la aplicación están centralizadas en `models.ts`. Ningún componente define sus propios tipos de dominio; todos los importan desde aquí.

2.1 interface BusinessProfile

■ src/app/shared/models.ts • línea 1-7

Representa el perfil de configuración de un negocio. Es la entidad central de la aplicación; prácticamente todos los componentes del dashboard la consumen.

Atributo / Propiedad	Tipo	Descripción
id	number	Identificador único del negocio.
businessName	string	Nombre visible del negocio. Se muestra en la página de reserva pública.
slug	string	Fragmento de URL amigable. Los clientes acceden en <code>/book/{slug}</code> .
timeZone	string	Zona horaria IANA (p. ej. "America/Bogota"). Usada para mostrar horarios correctamente.
requiresApproval	boolean	Si es true, las citas quedan en estado Pending hasta aprobación manual. Si es false, se confirman automáticamente.

Ejemplo del proyecto: src/app/shared/models.ts líneas 1-7

```
// models.ts – línea 1
export interface BusinessProfile {
  id: number;
  businessName: string;
  slug: string;
  timeZone: string;
  requiresApproval: boolean;
}
```

2.2 interface Service

■ src/app/shared/models.ts • línea 9-14

Describe un servicio ofrecido por el negocio (p. ej. "Consulta General", "Revisión Completa"). La duración en minutos determina los slots de tiempo que genera `BusinessService.getSlots()`.

Atributo / Propiedad	Tipo	Descripción
id	number	Identificador único del servicio.
name	string	Nombre descriptivo visible al cliente.

durationMinutes	number	Duración de la sesión en minutos. Mínimo recomendado: 5.
isActive	boolean	Controla si el servicio aparece en la página de reserva pública. Solo los activos se muestran.

Ejemplo del proyecto: `src/app/shared/models.ts` líneas 9-14

```
// models.ts — línea 9
export interface Service {
  id: number;
  name: string;
  durationMinutes: number;
  isActive: boolean;
}
```

2.3 interface AvailabilityWindow

■ `src/app/shared/models.ts` • línea 16-21

Representa un bloque de tiempo disponible en un día de la semana. Un negocio puede tener múltiples ventanas por día (p. ej. mañana y tarde).

Atributo / Propiedad	Tipo	Descripción
id	number	Identificador único de la ventana.
dayOfWeek	0 1 2 3 4 5 6	Día de la semana: 0 = Domingo, 1 = Lunes ... 6 = Sábado. Sigue el estándar de <code>Date.getDay()</code> .
startTime	string	Hora de inicio en formato "HH:mm" (24h). Ej: "08:00".
endTime	string	Hora de fin en formato "HH:mm" (24h). Ej: "17:00".

Ejemplo del proyecto: `src/app/shared/models.ts` líneas 16-21

```
// models.ts — línea 16
export interface AvailabilityWindow {
  id: number;
  dayOfWeek: 0 | 1 | 2 | 3 | 4 | 5 | 6; // 0 = Sunday ... 6 = Saturday
  startTime: string; // "HH:mm"
  endTime: string; // "HH:mm"
}
```

2.4 interface Appointment

■ src/app/shared/models.ts • línea 23-33

Representa una cita concreta entre un cliente y el negocio. Contiene tanto los datos del cliente como las marcas de tiempo en UTC y el estado del flujo de aprobación.

Atributo / Propiedad	Tipo	Descripción
id	number	Identificador único de la cita.
serviceId	number	FK al servicio (Service.id) que se reservó.
serviceName	string	Nombre del servicio en el momento de la reserva (desnormalizado para mostrar en la UI).
clientName	string	Nombre completo del cliente.
clientEmail	string	Correo electrónico del cliente.
clientPhone	string	Teléfono del cliente (opcional).
startUtc	string	Fecha y hora de inicio en formato ISO 8601 UTC. Ej: "2026-06-25T08:00:00Z".
endUtc	string	Fecha y hora de fin en formato ISO 8601 UTC.
status	'Pending' 'Confirmed' 'Declined' 'Cancelled'	Estado del ciclo de vida de la cita.

Ejemplo del proyecto: src/app/shared/models.ts líneas 23-33

```
// models.ts – línea 23
export interface Appointment {
  id: number;
  serviceId: number;
  serviceName: string;
  clientName: string;
  clientEmail: string;
  clientPhone: string;
  startUtc: string; // ISO 8601
  endUtc: string;
  status: 'Pending' | 'Confirmed' | 'Declined' | 'Cancelled';
}
```

Estados posibles de Appointment.status

Estado	Descripción	Transición válida
Pending	Cita solicitada, pendiente de revisión del negocio.	→ Confirmed, → Declined
Confirmed	Aprobada por el negocio o confirmada automáticamente.	→ Cancelled
Declined	Rechazada por el negocio.	— (terminal)
Cancelled	Cancelada (por el negocio o el sistema).	— (terminal)

2.5 interface Slot

■ src/app/shared/models.ts • línea 35-39

Representa un slot de tiempo disponible calculado por el servicio. Se muestra al cliente en el paso 2 del flujo de reserva. No es persistente: se genera en tiempo real a partir de las ventanas de disponibilidad.

Atributo / Propiedad	Tipo	Descripción
date	string	Fecha en formato "YYYY-MM-DD". Ej: "2026-06-25".
startTime	string	Hora de inicio en formato "HH:mm". Ej: "09:00".
endTime	string	Hora de fin calculada sumando durationMinutes al startTime.

3. Servicios (Services)

3.1 AuthService

■ src/app/auth.service.ts • línea 1-33

Selector: Inyectable globalmente (providedIn: 'root'). Gestiona el estado de autenticación usando localStorage y un signal de Angular para reactividad en toda la app.

Decorador

```
src/app/auth.service.ts línea 3-4
@Inject({ providedIn: 'root' })
export class AuthService { ... }
```

Propiedades

Atributo / Propiedad	Tipo	Descripción
TOKEN_KEY	private readonly string	Clave en localStorage donde se guarda el token. Valor: "dm_token".
isLoggedIn	Signal	Signal público. true cuando existe un token en localStorage. Reactivo: los componentes que lo lean se actualizan automáticamente al cambiar.

Métodos

Método / Función	Retorna	Descripción
login(email, password)	boolean	Valida que los parámetros no estén vacíos. En la implementación actual (mock) acepta cualquier credencial no vacía, almacena "mock-jwt-token" en localStorage y actualiza isLoggedIn.
register(email, password)	boolean	Idéntica lógica a login(). Registra y autentica al usuario en un solo paso.
logout()	void	Elimina el token de localStorage y pone isLoggedIn en false.
getToken()	string null	Devuelve el token almacenado o null si no hay sesión activa.

```
Ejemplo del proyecto: src/app/auth.service.ts líneas 8-16
// auth.service.ts – línea 8
isLoggedIn = signal(!localStorage.getItem(this.TOKEN_KEY));
// auth.service.ts – línea 10
login(email: string, password: string): boolean {
  if (!email.trim() || !password.trim()) return false;
  localStorage.setItem(this.TOKEN_KEY, 'mock-jwt-token');
  this.isLoggedIn.set(true);
  return true;
}
```


3.2 BusinessService

■ src/app/business.service.ts • línea 1-147

Servicio central de estado de la aplicación. Almacena y expone, mediante **signals** de Angular, todos los datos del negocio: perfil, servicios, ventanas de disponibilidad y citas. También contiene la lógica de generación de slots de tiempo disponibles.

Señales de estado (signals)

Atributo / Propiedad	Tipo	Descripción
profile	Signal	Estado del perfil del negocio. Inicializado con datos mock. Cualquier componente puede leer y el servicio es el único que escribe.
services	Signal	Lista de servicios del negocio. Inicializada con 3 servicios de ejemplo.
windows	Signal	Ventanas de disponibilidad semanal. Inicializadas con horarios de lunes a viernes.
appointments	Signal	Lista de todas las citas (mock inicial con 2 citas de ejemplo).

Propiedad privada

Atributo / Propiedad	Tipo	Descripción
nextId	private number	Contador auto-incremental que asigna IDs únicos a nuevos registros. Inicia en 100.

■ src/app/business.service.ts • línea 46

Métodos — Grupo: Perfil

Método / Función	Retorna	Descripción
updateProfile(p: BusinessProfile)	void	Reemplaza el perfil actual con una copia inmutable del parámetro. Usa spread para evitar mutaciones directas al objeto del signal.

```
Ejemplo del proyecto: src/app/business.service.ts línea 49-51
// business.service.ts — línea 49
updateProfile(p: BusinessProfile): void {
  this.profile.set({ ...p });
}
```

Métodos — Grupo: Servicios

Método / Función	Retorna	Descripción
addService(s: Omit)	void	Agrega un nuevo servicio a la lista asignándole un ID auto-generado con nextId++.

updateService(s: Service)	void	Reemplaza en la lista el servicio con el mismo id, dejando los demás intactos.
deleteService(id: number)	void	Filtra la lista eliminando el servicio con el id dado.

Ejemplo del proyecto: `src/app/business.service.ts` líneas 54-64

```
// business.service.ts — líneas 54-64
addService(s: Omit<Service, "id">): void {
  this.services.update(list => [...list, { ...s, id: this.nextId++ }]);
}
updateService(s: Service): void {
  this.services.update(list => list.map(x => x.id === s.id ? s : x));
}
deleteService(id: number): void {
  this.services.update(list => list.filter(x => x.id !== id));
}
```

Métodos — Grupo: Disponibilidad

Método / Función	Retorna	Descripción
addWindow(w: Omit)	void	Agrega una ventana de disponibilidad con ID auto-generado.
saveWindows(windows: AvailabilityWindow[])	void	Reemplaza toda la lista de ventanas (usado para guardar la edición masiva de la página de disponibilidad).
deleteWindow(id: number)	void	Elimina la ventana con el id dado.

Métodos — Grupo: Citas

Método / Función	Retorna	Descripción
approveAppointment(id: number)	void	Cambia el status de la cita a 'Confirmed'.
declineAppointment(id: number)	void	Cambia el status de la cita a 'Declined'.
cancelAppointment(id: number)	void	Cambia el status de la cita a 'Cancelled'.
addBooking(booking: Omit)	void	Registra una nueva cita (creada desde la página de reserva pública). Asigna ID auto-generado.

Método clave: getSlots()

■ src/app/business.service.ts • línea 101-146

Algoritmo que calcula los slots de tiempo disponibles para un servicio en los próximos N días. Es la pieza más compleja del sistema.

Atributo / Propiedad	Tipo	Descripción
serviceId	number	ID del servicio para el cual calcular slots.
days	number	Número de días hacia adelante a considerar (default: 14).

Retorna: Slot[] — arreglo de slots disponibles, sin conflictos con citas Pending o Confirmed existentes.

Algoritmo paso a paso:

- 1. Busca el servicio por serviceId. Si no existe, retorna [].
- 2. Itera sobre los próximos `days` días (a partir de hoy).
- 3. Para cada día obtiene el dayOfWeek (0-6) y las ventanas de disponibilidad de ese día.
- 4. Dentro de cada ventana genera slots consecutivos con duración = service.durationMinutes.
- 5. Por cada slot candidato verifica si hay conflicto con citas existentes (Pending o Confirmed).
- — Un conflicto ocurre cuando: slotStart < aEnd && slotEnd > aStart (overlap check).
- 6. Solo agrega el slot al resultado si NO hay conflicto.

```
Ejemplo del proyecto: src/app/business.service.ts líneas 129-134
// business.service.ts – líneas 129-134 (verificación de conflictos)
const hasConflict = this.appointments().some(a => {
  if (a.status === 'Declined' || a.status === 'Cancelled') return false;
  const aStart = new Date(a.startUtc).getTime();
  const aEnd = new Date(a.endUtc).getTime();
  return slotStart < aEnd && slotEnd > aStart;
});
```

4. Componentes

4.1 App — Componente Raíz

■ src/app/app.ts • línea 1-12

Atributo / Propiedad	Tipo	Descripción
selector	'app-root'	Selector CSS del componente raíz. Referenciado en src/index.html como .
template	' '	Template inline. Solo renderiza el outlet del router; toda la UI viene de las rutas.
styleUrl	'./app.scss'	Hoja de estilos específica del componente raíz.
title	Signal	Signal con el nombre de la aplicación. Valor: "DateManager.FrontEnd". No se usa en el template actualmente.

```
Ejemplo del proyecto: src/app/app.ts líneas 4-12
// app.ts – línea 4
@Component({
  selector: 'app-root',
  imports: [RouterOutlet],
  template: '<router-outlet/>',
  styleUrl: './app.scss'
})
export class App {
  protected readonly title = signal('DateManager.FrontEnd');
}
```

4.2 LayoutComponent — Shell del Dashboard

■ src/app/shared/layout/layout.component.ts • línea 1-21

Componente "shell" que envuelve todas las páginas del dashboard autenticado. Renderiza la barra lateral de navegación y el para las sub-páginas.

Atributo / Propiedad	Tipo	Descripción
selector	'app-layout'	Selector del componente.
imports	RouterOutlet, RouterLink, RouterLinkActive, DateManagerIconComponent	Módulos y componentes standalone que necesita el template.
auth	private AuthService	Inyección del servicio de autenticación (inject pattern).

router	<code>private Router</code>	Inyección del router de Angular.
Método / Función	Retorna	Descripción
<code>logout()</code>	<code>void</code>	Llama a <code>auth.logout()</code> y navega a <code>/Login</code> . Invocado desde el botón "Cerrar sesión" del sidebar.
Ejemplo del proyecto: <code>src/app/shared/layout/layout.component.html</code> líneas 25-27 <pre>// layout.component.html – líneas 25-27 <button class="sidebar-link" (click)="logout()"> <DateManager-icon name="logout" /> Cerrar sesión </button></pre>		

4.3 DateManagerIconComponent — Icono Reutilizable

■ `src/app/shared/DateManager-icon/DateManager-icon.component.ts` • línea 1-32

Componente utilitario que mapea nombres semánticos a nombres de iconos de Material Icons. Evita que los componentes dependan directamente del sistema de iconos, facilitando un cambio futuro.

Atributo / Propiedad	Tipo	Descripción
<code>selector</code>	<code>'DateManager-icon'</code>	Selector del componente. Se usa directamente en HTML como etiqueta.
<code>name</code>	<code>@Input() string</code>	Nombre semántico del icono. Se traduce a través de <code>ICON_MAP</code> .
<code>size</code>	<code>@Input() number null</code>	Tamaño en píxeles del ícono. Default: 18px (aplicado via style binding).
<code>ICON_MAP</code>	<code>const Record</code>	Mapeo estático: nombre semántico → nombre de clase Material Icons. Definido en la línea 3.
Método / Función	Retorna	Descripción
<code>iconName (getter)</code>	<code>string</code>	Propiedad computada que retorna <code>ICON_MAP[this.name]</code> o el nombre directo si no está en el mapa.

Ejemplo del proyecto: `src/app/shared/DateManager-icon/DateManager-icon.component.ts` líneas 3-19

```
// DateManager-icon.component.ts – línea 3
const ICON_MAP: Record<string, string> = {
  user: 'person',
  ext: 'lock',
  arrow: 'arrow_forward',
  alert: 'warning',
  calendar: 'calendar_month',
  service: 'design_services',
  clock: 'schedule',
  check: 'check_circle',
  close: 'cancel',
  edit: 'edit',
  delete: 'delete',
  add: 'add',
  logout: 'logout',
  business: 'business',
  menu: 'menu',
};

// Uso en template (layout.component.html – línea 11):
<DateManager-icon name="calendar" />
```

4.4 LoginPage — Página de Inicio de Sesión

■ src/app/login-page/login-page.ts • línea 1-38

Atributo / Propiedad	Tipo	Descripción
selector	'app-login-page'	Selector del componente.
imports	FormsModule, NgIf, DateManagerIconModule	Dependencias standalone.
auth	private AuthService	Servicio de autenticación inyectado.
router	Router	Router inyectado (público para acceso desde el template).
username	string	Campo de texto ligado por ngModel al input de usuario.
password	string	Campo de texto ligado por ngModel al input de contraseña.
err	string	Mensaje de error a mostrar en la UI. Vacío cuando no hay error.
loading	Signal	Controla el estado de carga del botón submit.

Método / Función	Retorna	Descripción
submit()	void	Valida campos no vacíos, llama a auth.login() y navega a /dashboard si es exitoso, o muestra mensaje de error.

Ejemplo del proyecto: src/app/login-page/login-page.html líneas 32-43

```
// login-page.html – línea 32
<div *ngIf="err" class="login-err">
  <DateManager-icon name="alert" [size]="14" />
  <span>{{ err }}</span>
</div>

// login-page.html – línea 37
<button class="btn primary lg block" type="submit" [disabled]="loading()">
  <span *ngIf="loading()">Verificando...</span>
  <ng-container *ngIf="!loading()">
    Ingresar <DateManager-icon name="arrow" />
  </ng-container>
</button>
```

4.5 RegistrationPage — Página de Registro

■ src/app/registration-page/registration-page.ts • línea 1-66

Extiende el flujo de login con validación de contraseña robusta y un indicador visual de fortaleza.

Atributo / Propiedad	Tipo	Descripción
selector	'app-registration-page'	Selector del componente.
username	string	Campo email/usuario.
password	string	Campo contraseña.
confirmPassword	string	Campo de confirmación.
err	string	Mensaje de error de validación.
loading	Signal	Estado del botón submit.
passwordStrength	'weak' 'medium' 'strong' ''	Estado de fortaleza calculado al escribir la contraseña.
StrongPasswordRegx	const RegExp	Regex que exige: mínimo 8 caracteres, una mayúscula, una minúscula, un dígito. Definida en línea 8.

Método / Función	Retorna	Descripción
onPasswordChange()	void	Se activa en cada pulsación del campo password. Calcula un score 0-5 basado en: longitud≥8, mayúscula, minúscula, dígito, carácter especial. Asigna passwordStrength según el score.
validateSamePassword()	void	Compara password con confirmPassword y asigna un mensaje de error si no coinciden.
submit()	void	Valida campos, aplica la regex de contraseña fuerte y comprueba coincidencia. Llama a auth.register() y redirige a /Login.

Ejemplo del proyecto: src/app/registration-page/registration-page.ts líneas 8, 27-38

```
// registration-page.ts – línea 8
const StrongPasswordRegx = /^(?=.*[A-Z])(?=.*[a-z])(?=.*\d){8,}$/;
// registration-page.ts – líneas 27-38 (cálculo de fortaleza)
onPasswordChange() {
  const p = this.password;
  if (!p) { this.passwordStrength = ""; return; }
  let score = 0;
  if (p.length >= 8) score++;
  if (/[A-Z]/.test(p)) score++;
  if (/[a-z]/.test(p)) score++;
  if (/^\d/.test(p)) score++;
  if (/^[A-Za-z0-9]/.test(p)) score++;
  if (score <= 2) this.passwordStrength = "weak";
  else if (score <= 3) this.passwordStrength = "medium";
  else this.passwordStrength = "strong";
}
```


4.6 AppointmentsPage — Gestión de Citas

■ src/app/appointments-page/appointments-page.ts • línea 1-47

Página del dashboard que lista, filtra y permite gestionar el estado de todas las citas del negocio.

Atributo / Propiedad	Tipo	Descripción
selector	'app-appointments-page'	Selector.
svc	BusinessService	Servicio inyectado para acceder a citas.
activeTab	Signal	Pestaña activa: 'all' 'pending' 'confirmed'. Valor inicial: 'all'.
filteredAppointments	computed Signal	Signal derivado. Filtra svc.appointments() según activeTab(). Se recalcula automáticamente cuando cambia cualquiera de los dos.

Type alias: type Tab = 'all' | 'pending' | 'confirmed' — definido en línea 4 para tipar la pestaña activa.

Método / Función	Retorna	Descripción
setTab(tab: Tab)	void	Actualiza activeTab. Dispara la recomputación de filteredAppointments.
approve(id: number)	void	Delega a svc.approveAppointment(id).
decline(id: number)	void	Delega a svc.declineAppointment(id).
cancel(id: number)	void	Delega a svc.cancelAppointment(id).
formatDate(iso: string)	string	Convierte una fecha ISO a string localizado en español (Colombia). Ej: "25 jun. 2026, 8:00 a. m.".
statusLabel(status)	string	Traduce el status en inglés al español para mostrar en la UI.
statusClass(status)	string	Convierte el status a minúsculas para usarlo como clase CSS en el badge.

```
Ejemplo del proyecto: src/app/appointments-page/appointments-page.ts líneas 16-22
// appointments-page.ts – líneas 16-22
filteredAppointments = computed(() => {
  const all = this.svc.appointments();
  const tab = this.activeTab();
  if (tab === 'pending') return all.filter(a => a.status === 'Pending');
  if (tab === 'confirmed') return all.filter(a => a.status === 'Confirmed');
  return all;
});
// appointments-page.html – línea 33
<span [class]="`badge ` + statusClass(a.status)">{{ statusLabel(a.status) }}</span>
```

4.7 ServicesPage — Gestión de Servicios

■ src/app/services-page/services-page.ts • línea 1-48

Atributo / Propiedad	Tipo	Descripción
selector	'app-services-page'	Selector.
svc	BusinessService	Servicio inyectado.
showForm	Signal	Controla la visibilidad del formulario de agregar/editar.
editingService	Service null	Servicio que se está editando. null cuando se está creando uno nuevo.
form	Omit	Datos del formulario. Inicializado con { name: "", durationMinutes: 30, isActive: true }.

Método / Función	Retorna	Descripción
openAdd()	void	Limpia editingService, resetea form a valores por defecto y muestra el formulario.
openEdit(s: Service)	void	Carga los datos de s en form, asigna editingService = s y muestra el formulario.
save()	void	Si editingService existe llama a svc.updateService(); si no, llama a svc.addService(). Oculta el formulario al finalizar.
cancel()	void	Oculta el formulario sin guardar.
delete(id: number)	void	Pide confirmación al usuario y llama a svc.deleteService(id).

4.8 AvailabilityPage — Disponibilidad Horaria

■ src/app/availability-page/availability-page.ts • línea 1-56

Permite al negocio configurar sus horarios de atención semanales. Opera sobre una copia local (localWindows) y solo persiste al presionar "Guardar".

Atributo / Propiedad	Tipo	Descripción
selector	'app-availability-page'	Selector.
days	{ dow: number; label: string }[]	Array constante DAYS con los 7 días de la semana en español. Definido en línea 6.
localWindows	AvailabilityWindow []	Copia editable de las ventanas. No modifica el signal hasta que el usuario guarda.
saved	Signal	Controla el indicador "✓ Guardado" que desaparece a los 2.5 segundos.
tmpId	private number	Contador para IDs temporales de ventanas nuevas. Inicia en 1000.

Método / Función	Retorna	Descripción
windowsForDay(dow: number)	AvailabilityWindow []	Filtra localWindows por el día de la semana dado.
addWindow(dow: number)	void	Agrega una ventana con horario default 09:00-17:00 para el día indicado.
deleteWindow(id: number)	void	Elimina la ventana de localWindows por id.
save()	void	Persiste localWindows en el signal del servicio mediante svc.saveWindows(). Activa el indicador de guardado por 2.5s.

```
Ejemplo del proyecto: src/app/availability-page/availability-page.ts líneas 51-55
// availability-page.ts — líneas 51-55
save(): void {
  this.svc.saveWindows(this.localWindows);
  this.saved.set(true);
  setTimeout(() => this.saved.set(false), 2500);
}
```

4.9 BusinessProfilePage — Perfil del Negocio

■ src/app/business-profile-page/business-profile-page.ts • línea 1-35

Atributo / Propiedad	Tipo	Descripción
selector	'app-business-profile-page'	Selector.

svc	<code>private BusinessService</code>	Servicio inyectado.
profile	<code>BusinessProfile</code>	Copia local del perfil para edición. Inicializada con spread del signal ({...svc.profile()}).
timezones	<code>string[]</code>	Array constante TIMEZONES con 7 zonas horarias de América y Europa. Definido en línea 6.
saved	<code>Signal</code>	Indicador visual de guardado exitoso.

Método / Función	Retorna	Descripción
<code>save()</code>	<code>void</code>	Persiste la copia local al signal mediante <code>svc.updateProfile()</code> . Activa el indicador de guardado por 2.5s.

```
Ejemplo del proyecto: src/app/business-profile-page/business-profile-page.ts líneas 6-14
// business-profile-page.ts – líneas 6-14
const TIMEZONES = [
  'America/Bogota',
  'America/Mexico_City',
  'America/Lima',
  'America/Santiago',
  'America/Argentina/Buenos_Aires',
  'America/New_York',
  'Europe/Madrid',
];
```

4.10 BookingPage — Reserva Pública (4 pasos)

■ src/app/booking-page/booking-page.ts • línea 1-93

Página pública accesible sin autenticación. Implementa un wizard de 4 pasos: (1) Selección de servicio → (2) Elección de horario → (3) Datos del cliente → (4) Confirmación. El slug del negocio se pasa como parámetro de ruta.

Type alias: `type Step = 'service' | 'slot' | 'form' | 'success'` — definido en línea 6.

Atributo / Propiedad	Tipo	Descripción
selector	'app-booking-page'	Selector.
svc	private BusinessService	Servicio inyectado para leer datos y crear reservas.
step	Signal	Paso actual del wizard. Valor inicial: 'service'.
selectedService	Signal	Servicio seleccionado por el cliente en el paso 1.
selectedSlot	Signal	Slot seleccionado por el cliente en el paso 2.
clientName	string	Campo del formulario del paso 3.
clientEmail	string	Campo del formulario del paso 3.
clientPhone	string	Campo del formulario del paso 3 (opcional).
err	string	Mensaje de error del paso 3.
loading	Signal	Estado de envío del formulario.
profile	Signal	Referencia directa al signal de perfil del servicio.
services	computed Signal	Solo los servicios activos (isActive === true).
slots	computed Signal	Slots calculados para el servicio seleccionado (getSlots(id, 14)).
slotGroups	computed Signal<date, slots>[]>	Slots agrupados por fecha para renderizado en el template.

Método / Función	Retorna	Descripción
stepNumber()	number	Mapea el step actual a un número 1-4 para el indicador de progreso.
selectService(s: Service)	void	Guarda el servicio y avanza al paso 'slot'.
selectSlot(slot: Slot)	void	Guarda el slot y avanza al paso 'form'.
goBack()	void	Retrocede un paso: form→slot o slot→service.
submit()	void	Valida nombre y email, crea la reserva con addBooking() y avanza al paso 'success'. El status depende de profile().requiresApproval.
formatDate(dateStr)	string	Convierte 'YYYY-MM-DD' a fecha larga en español (es-CO). Ej: 'jueves, 25 de junio de 2026'.

```
Ejemplo del proyecto: src/app/booking-page/booking-page.ts líneas 73-84
// booking-page.ts — líneas 73-84 (submit)
this.svc.addBooking({
  serviceId: service.id,
  serviceName: service.name,
  clientName: this.clientName,
  clientEmail: this.clientEmail,
  clientPhone: this.clientPhone,
  startUtc: `${slot.date}T${slot.startTime}:00Z`,
  endUtc: `${slot.date}T${slot.endTime}:00Z`,
  status: this.svc.profile().requiresApproval ? 'Pending' : 'Confirmed',
});
```

5. Guardia de Ruta y Enrutamiento

5.1 authGuard — Guardia de Autenticación

■ src/app/auth.guard.ts • línea 1-10

Guard funcional (CanActivateFn) que protege las rutas del dashboard. Si el usuario no está autenticado, redirige a /Login sin mostrar la página solicitada.

Atributo / Propiedad	Tipo	Descripción
authGuard	CanActivateFn	Función exportada que Angular Router invoca antes de activar cada ruta protegida. Inyecta AuthService y Router mediante inject().

Ejemplo del proyecto: src/app/auth.guard.ts líneas 5-10

```
// auth.guard.ts — líneas 5-10
export const authGuard: CanActivateFn = () => {
  const auth = inject(AuthService);
  const router = inject(Router);
  if (auth.isLoggedIn()) return true;
  return router.createUrlTree(['/Login']);
};
```

5.2 Rutas (app.routes.ts)

■ src/app/app.routes.ts • línea 1-42

Todas las rutas usan **lazy loading** mediante loadComponent. Esto reduce el bundle inicial al cargar cada componente solo cuando se navega a él.

Ruta	Componente	Guard	Descripción
/Login	LoginPage	—	Página de inicio de sesión.
/Registration	RegistrationPage	—	Página de registro de nuevo usuario.
/dashboard	LayoutComponent	authGuard	Shell del dashboard. Protegida. Redirige a /dashboard/appointments por defecto.
/dashboard/appointments	AppointmentsPage	heredada	Gestión de citas.
/dashboard/services	ServicesPage	heredada	Gestión de servicios.
/dashboard/availability	AvailabilityPage	heredada	Configuración de horarios.
/dashboard/profile	BusinessProfilePage	heredada	Perfil del negocio.
/book/:slug	BookingPage	—	Reserva pública. Pública, sin autenticación.
/ (vacío)	redirige a /Login	—	Redirección de la ruta raíz.

5.3 app.config.ts — Configuración de la Aplicación

■ src/app/app.config.ts • línea 1-12

Punto de entrada de los providers de Angular. Al usar standalone components sin NgModule, la configuración global se centraliza aquí.

Atributo / Propiedad	Tipo	Descripción
appConfig	ApplicationConfig	Objeto de configuración exportado. Consumido por bootstrapApplication() en main.ts.
provideBrowserGlobalErrorListeners()	Provider	Registra listeners globales de errores del navegador.
provideRouter(routes)	Provider	Configura el router de Angular con las rutas definidas en app.routes.ts.
provideHttpClient()	Provider	Habilita el HttpClient para peticiones HTTP (preparado para la integración con el backend).

6. Estilos Globales — src/styles.scss

El archivo `styles.scss` define el sistema de diseño completo mediante variables CSS (**custom properties**). Todos los componentes las consumen sin importar el archivo, gracias al alcance global de `:root`.

6.1 Variables de Color

■ src/styles.scss • línea 2-16		
Variable	Valor	Uso
--primary	#4f46e5	Color primario (indigo). Botones, links, acentos principales.
--primary-hover	#4338ca	Variante oscura del primario para el estado :hover.
--bg	#f8fafc	Fondo global de la aplicación (gris muy claro).
--card-bg	#ffffff	Fondo de tarjetas y formularios.
--border	#e2e8f0	Color de bordes y divisores.
--text	#0f172a	Color de texto principal (slate-950).
--text-muted	#64748b	Texto secundario / placeholders / etiquetas.
--fg-muted	#64748b	Alias de --text-muted (compatibilidad con registration-page).
--error	#ef4444	Color de error (rojo).
--success	#22c55e	Color de éxito (verde).
--warning	#eab308	Color de advertencia (amarillo).
--pending-color	#f97316	Color específico para el estado Pendiente de una cita.

6.2 Variables de Forma y Sombra

■ src/styles.scss • línea 17-19		
Atributo / Propiedad	Tipo	Descripción
--radius	8px	Radio de borde estándar para tarjetas, inputs y botones.
--radius-lg	12px	Radio de borde grande para modales y paneles.
--shadow	0 1px 3px rgba(0,0,0,.10)...	Sombra suave para elevación de tarjetas.
--shadow-lg	0 10px 15px -3px rgba(0,0,0,.10)...	Sombra pronunciada para modales y overlays.

6.3 Variables de Tipografía

■ src/styles.scss • línea 21-24		
Atributo / Propiedad	Tipo	Descripción

--text-sm	0.875rem (14px)	Texto pequeño: metadatos, etiquetas, notas.
--text-md	1rem (16px)	Texto normal: cuerpo de página.
--text-lg	1.125rem (18px)	Texto grande: títulos de sección.
--text-xl	1.25rem (20px)	Texto extra grande: títulos principales.

6.4 Reset CSS y Fuente Global

■ src/styles.scss • línea 27-35

Se aplica un reset de box-sizing y márgenes/paddings globales. La fuente principal es **Inter** con fallback a system-ui.

```
Ejemplo del proyecto: src/styles.scss líneas 27-35
/* styles.scss — líneas 27-35 */
*, *::before, *::after { box-sizing: border-box; margin: 0; padding: 0; }
body {
  font-family: 'Inter', system-ui, -apple-system, sans-serif;
  background: var(--bg);
  color: var(--text);
  line-height: 1.5;
}
```

7. Clases CSS Reutilizables

Las siguientes clases CSS se definen en los archivos SCSS de los componentes pero se usan de forma coherente en toda la aplicación como si fueran un sistema de diseño.

7.1 Botones

Clase	Descripción
<code>.btn</code>	Clase base de botón. Define padding, border-radius, cursor, transición.
<code>.btn.primary</code>	Botón de acción principal. Fondo: <code>var(--primary)</code> , texto blanco.
<code>.btn.secondary</code>	Botón de acción secundaria. Borde con <code>var(--border)</code> , fondo transparente.
<code>.btn.danger</code>	Botón destructivo. Color: <code>var(--error)</code> .
<code>.btn.sm</code>	Modificador de tamaño: padding y font-size reducidos.
<code>.btn.lg</code>	Modificador de tamaño: padding y font-size grandes.
<code>.btn.block</code>	Ocupa el 100% del ancho del contenedor.

Ejemplos de uso de botones en el proyecto

```
<!-- Uso real en appointments-page.html – línea 38 -->
<button class="btn primary sm" (click)="approve(a.id)">Aprobar</button>
<button class="btn danger sm" (click)="decline(a.id)">Rechazar</button>

<!-- Uso real en login-page.html – línea 37 -->
<button class="btn primary lg block" type="submit" [disabled]="loading()">
```

7.2 Formularios

Clase	Descripción
<code>.field</code>	Contenedor de un campo de formulario (label + input).
<code>.input</code>	Estilo base de input/select: borde, border-radius, padding, width: 100%.
<code>.input.invalid</code>	Aplica borde rojo (<code>var(--error)</code>) para feedback de validación.
<code>.input-group</code>	Wrapper flex que coloca un icono a la izquierda del input.
<code>.form-row</code>	Fila con dos columnas de campos (responsive).
<code>.form-section</code>	Tarjeta blanca (<code>card-bg</code>) con sombra y padding que agrupa un formulario.
<code>.form-actions</code>	Fila de botones de acción (guardar/cancelar) alineados a la derecha.

7.3 Badges de Estado

■ `src/app/appointments-page/appointments-page.html` • línea 33

Los badges de estado combinan la clase base `.badge` con una clase modificadora que corresponde al status en minúsculas.

Atributo / Propiedad	Tipo	Descripción
----------------------	------	-------------

<code>.badge</code>	CSS class	Clase base: pill redondeado con padding y font-size pequeño.
<code>.badge.pending</code>	CSS class	Fondo naranja claro, texto naranja. Estado: Pendiente.
<code>.badge.confirmed</code>	CSS class	Fondo verde claro, texto verde. Estado: Confirmada.
<code>.badge.declined</code>	CSS class	Fondo rojo claro, texto rojo. Estado: Rechazada.
<code>.badge.cancelled</code>	CSS class	Fondo gris claro, texto gris. Estado: Cancelada.

Ejemplo del proyecto: uso de badges con clases dinámicas

```
<!-- appointments-page.html – línea 33 -->
<span [class]='badge ' + statusClass(a.status)'>{{ statusLabel(a.status) }}</span>
<!-- services-page.html – línea 53 -->
<span class="badge" [class.confirmed]="s.isActive" [class.cancelled]="!s.isActive">
  {{ s.isActive ? "Activo" : "Inactivo" }}
</span>
```

7.4 Layout del Dashboard

■ src/app/shared/layout/layout.component.html • línea 1-33

Atributo / Propiedad	Tipo	Descripción
<code>.dashboard-layout</code>	CSS class	Contenedor flex raíz: sidebar fijo + main scrollable.
<code>.sidebar</code>	CSS class	Barra lateral izquierda: fondo indigo, navegación vertical.
<code>.sidebar-brand</code>	CSS class	Área del logo en la parte superior del sidebar.
<code>.sidebar-link</code>	CSS class	Enlace de navegación del sidebar con hover state.
<code>.sidebar-spacer</code>	CSS class	Flex-grow: empuja el botón de logout al fondo del sidebar.
<code>.dashboard-main</code>	CSS class	Área de contenido principal con padding y scroll.
<code>.page-header</code>	CSS class	Cabecera de cada sub-página: título h1 + descripción.

7.5 Clases de la Página de Login

■ src/app/login-page/login-page.html • línea 1-69

Atributo / Propiedad	Tipo	Descripción
<code>.login-bg</code>	CSS class	Contenedor flex 100vw/100vh. Divide en formulario (izquierda) y panel decorativo (derecha).
<code>.login-card</code>	CSS class	Tarjeta blanca centrada con el formulario. Max-width fijo.
<code>.login-brand</code>	CSS class	Área del logo y slogan de la tarjeta.
<code>.login-tag</code>	CSS class	Subtítulo "Sistema Integral de Gestión de Citas".

<code>.login-form</code>	CSS class	Contenedor del formulario con gap entre campos.
<code>.login-err</code>	CSS class	Bloque de error rojo con icono y texto.
<code>.login-foot</code>	CSS class	Pie de la tarjeta con link de ayuda.
<code>.login-aside</code>	CSS class	Panel decorativo derecho con gradiente indigo.
<code>.login-hero</code>	CSS class	Contenedor centrado dentro del panel decorativo.
<code>.datemanager-logo</code>	CSS class	Wrapper flex del logotipo (mark + wordmark).
<code>.datemanager-mark</code>	CSS class	Ícono geométrico del logo.
<code>.datemanager-word</code>	CSS class	Texto "DateManager" del logo.

8. Directivas y Bindings de Angular usados en el proyecto

A continuación se documentan todas las directivas y sintaxis de binding de Angular utilizadas en los templates del proyecto, con ejemplos concretos.

Directiva / Binding	Tipo	Descripción + Ejemplo del proyecto
<code>[(ngModel)]</code>	Two-way binding	Liga un campo de formulario a una propiedad del componente en ambas direcciones. <code>[(ngModel)]="username"</code> (login-page.html:17)
<code>[class.xxx]</code>	Class binding	Agrega/quita una clase CSS basada en una expresión booleana. <code>[class.invalid]="err && !username.trim()"</code> (login-page.html:18)
<code>[class]</code>	Class binding (completo)	Evalúa una expresión string y asigna el resultado como clase CSS. <code>[class]='badge ' + statusClass(a.status)"</code> (appointments-page.html:33)
<code>[disabled]</code>	Property binding	Habilita/deshabilita un elemento nativo según la expresión. <code>[disabled]="loading()"</code> (booking-page.html:107)
<code>[value]</code>	Property binding	Asigna el atributo value de un de forma dinámica. <code>[value]="tz"</code> (business-profile-page.html:27)
<code>[style.fontSize.px]</code>	Style binding con unidad	Asigna un valor numérico a una propiedad CSS con unidad automática. <code>[style.fontSize.px]="size ?? 18"</code> (DateManager-icon.component.ts:23)
<code>(click)</code>	Event binding	Escucha el evento click del DOM y llama al método indicado. <code>(click)="approve(a.id)"</code> (appointments-page.html:38)
<code>(ngSubmit)</code>	Event binding de formulario	Escucha el evento submit del Angular. <code>(ngSubmit)="submit()"</code> (login-page.html:11)
<code>(input)</code>	Event binding de input	Se dispara en cada cambio del valor de un input. <code>(input)="onPasswordChange()"</code> (registration-page.html:29)
<code>*ngIf</code>	Directiva estructural (NgIf)	Agrega/elimina el elemento del DOM según la condición. <code>*ngIf="err"</code> (login-page.html:32)

@if	Flow control (Angular 17+)	Alternativa moderna a *ngIf. Renderizado condicional en template. <code>@if (step() !== "success") { ... }</code> (booking-page.html:14)
@for ... track	Flow control (Angular 17+)	Alternativa moderna a *ngFor. Itera sobre un array. track es requerido para optimizar el DOM. <code>@for (s of services(); track s.id) { ... }</code> (booking-page.html:28)
@empty	Flow control (Angular 17+)	Bloque opcional dentro de @for. Se renderiza cuando el array está vacío. <code>@empty { No hay citas... }</code> (appointments-page.html:47)
routerLink	Directiva de router	Navega a una ruta sin recargar la página. <code>routerLink="appointments"</code> (layout.component.html:10)
routerLinkActive	Directiva de router	Agrega una clase CSS cuando la ruta del routerLink está activa. <code>routerLinkActive="active"</code> (layout.component.html:10)
	Componente de router	Punto de inserción donde el router renderiza el componente de la ruta activa. (layout.component.html:31)

9. Sistema de Señales (Signals) de Angular

El proyecto usa exclusivamente el sistema de señales de Angular (introducido en v16) en lugar de RxJS Observables para la gestión de estado. Esto simplifica el código al eliminar la necesidad de suscripciones y pipes.

API	Tipo retorno	Descripción + Ejemplo
<code>signal(value)</code>	<code>WritableSignal</code>	Crea una señal con un valor inicial. <code>isLoggedIn = signal(!!localStorage.getItem(this.TOKEN_KEY));</code> (auth.service.ts:8)
<code>.set(value)</code>	<code>void</code>	Reemplaza el valor de la señal. <code>this.isLoggedIn.set(true);</code> (auth.service.ts:14)
<code>.update(fn)</code>	<code>void</code>	Actualiza el valor usando una función que recibe el valor actual. <code>this.services.update(list => [...list, { ...s, id: this.nextId++ }]);</code> (business.service.ts:55)
<code>computed(() =>...)</code>	<code>Signal (read-only)</code>	Crea una señal derivada que se recalcula cuando sus dependencias cambian. <code>filteredAppointments = computed(() => { ... });</code> (appointments-page.ts:16)
<code>signal()</code> en template	<code>T</code>	En el template se invoca la señal como función para leer su valor: <code>signal()</code> . <code>[class.active]="activeTab() === 'all'"</code> (appointments-page.html:8)

10. Flujos de Usuario Principales

10.1 Flujo de Autenticación

#	Actor	Acción
1	Usuario accede a /	App redirige a /Login (app.routes.ts:41)
2	LoginPage: submit()	Valida campos → llama AuthService.login()
3	AuthService.login()	Guarda token en localStorage, sets isLoggedIn=true
4	Router navega a /dashboard	authGuard verifica isLoggedIn() → permite acceso
5	LayoutComponent se carga	Muestra sidebar + AppointmentsPage por defecto
6	logout()	AuthService.logout() → localStorage limpio → /Login

10.2 Flujo de Reserva Pública (/book/:slug)

#	Acción	Resultado
1	Cliente accede a /book/mi-negocio	BookingPage se carga. step = "service"
2	selectService(s)	selectedService = s, step = "slot"
3	slots computed()	getSlots(s.id, 14) calcula slots disponibles sin conflictos
4	selectSlot(slot)	selectedSlot = slot, step = "form"
5	submit()	Valida nombre+email, llama addBooking() con status=Pending/Confirmed según requiresApproval
6	step = "success"	Se muestra confirmación. Si requiresApproval: "El negocio la revisará..."

11. Índice de Archivos y Secciones

Archivo	Contenido principal	Sección doc.
src/app/shared/models.ts	Interfaces de dominio: BusinessProfile, Service, AvailabilityWindow, Appointment, Slot	Sección 2
src/app/auth.service.ts	AuthService: login, logout, isLoggedIn signal	Sección 3.1
src/app/business.service.ts	BusinessService: estado central, getSlots(), CRUD de servicios y citas	Sección 3.2
src/app/auth.guard.ts	authGuard: CanActivateFn, protección de rutas del dashboard	Sección 5.1
src/app/app.routes.ts	Configuración de rutas con lazy loading	Sección 5.2
src/app/app.config.ts	ApplicationConfig: provideRouter, provideHttpClient	Sección 5.3
src/app/app.ts	App (componente raíz): template mínimo con	Sección 4.1
src/app/shared/layout/layout.component	LayoutComponent: shell con sidebar y logout	Sección 4.2
src/app/shared/DateManager-icon/...	DateManagerIconComponent: wrapper de Material Icons	Sección 4.3
src/app/login-page/	LoginPage: formulario de autenticación	Sección 4.4
src/app/registration-page/	RegistrationPage: registro con validación de contraseña	Sección 4.5
src/app/appointments-page/	AppointmentsPage: tabla de citas con filtros y acciones	Sección 4.6
src/app/services-page/	ServicesPage: CRUD de servicios con formulario inline	Sección 4.7
src/app/availability-page/	AvailabilityPage: editor de ventanas de disponibilidad semanal	Sección 4.8
src/app/business-profile-page/	BusinessProfilePage: formulario de perfil con selector de zona horaria	Sección 4.9
src/app/booking-page/	BookingPage: wizard de reserva pública en 4 pasos	Sección 4.10
src/styles.scss	Variables CSS globales, reset, fuente Inter	Sección 6